

Trabajo Práctico N.º 1 — Food Store

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Materia	Base de Datos I — Tecnicatura Universitaria en Programación (UTN)
Proyecto	Food Store — sistema de gestión de pedidos (Semana 1)
Desarrollo	Parte 1 (modelo ER), Parte 2 (modelo relacional) y Parte 3 (normalización)
Anexo	Parte 4 (schema.sql) — entregada como archivo <code>schema.sql</code> aparte

Parte 1 — Modelo entidad-relación

1.1 Notación elegida

Se utiliza notación **Pata de Gallo (Crow's Foot)**, sostenida en todo el diagrama. El diagrama se entregó aparte como `diagrama_ER.png` (y `diagrama_ER.pdf`), construido con Mermaid: la fuente editable es `diagrama_ER.mmd` (pegable en mermaid.live) y `diagrama_ER.html` lo renderiza en el navegador. Se incluye además `alternativa_dbdiagram_io.dbml` por si se prefiere regenerar la misma notación en dbdiagram.io.

1.2 Diccionario de entidades y atributos

Los atributos marcados como *adicional razonable* no están explícitos en el enunciado y se agregan porque el negocio los usa en la práctica (según lo autoriza la consigna).

Entidad	Atributo	Tipo conceptual	Clave	Nulabilidad	Regla / Nota
CLIENTE	id	numérico (entero)	PK	NOT NULL	Clave elegida; autogenerada.
	nombre	texto	—	NOT NULL	Separado de apellido (pregunta guía 3).
	apellido	texto	—	NOT NULL	Permite ordenar/buscar por apellido.
	email	texto	UK	NOT NULL	R6: identifica de forma única al cliente; clave candidata alternativa.

	telefono	texto	—	NULL	Adicional razonable (dato de contacto opcional).

	created_at	fecha y hora c/ zona	—	NOT NULL	Fecha de alta (valor por defecto: ahora).
CATEGORIA	id	numérico (entero)	PK	NOT NULL	Clave elegida.
	nombre	texto	UK	NOT NULL	Decisión de diseño: sin duplicados de presentación.
	descripcion	texto	—	NULL	Adicional razonable.
	activo	booleano	—	NOT NULL	R7 : baja lógica; default TRUE.
	created_at	fecha y hora c/ zona	—	NOT NULL	Default now().
PRODUCTO	id	numérico (entero)	PK	NOT NULL	Clave elegida.
	nombre	texto	—	NOT NULL	Sin unicidad declarada (no la exige ninguna regla).
	descripcion	texto	—	NULL	Adicional razonable.
	precio	numérico (monetario)	—	NOT NULL	R5 : no negativo; precio de lista actual.
	stock	numérico (entero)	—	NOT NULL	R5 : no negativo; default 0.
	categoria_id	numérico (entero)	FK → CATEGORIA	NOT NULL	R1 : participación total.
	activo	booleano	—	NOT NULL	R7 : baja lógica; default TRUE.

	created_at	fecha y hora c/ zona	—	NOT NULL	Default now().
PEDIDO	id	numérico (entero)	PK	NOT NULL	Clave elegida.
	fecha	fecha y hora c/ zona	—	NOT NULL	Momento del pedido; default now().
	cliente_id	numérico (entero)	FK → CLIENTE	NOT NULL	R2: participación total.
	forma_pago	enumerado	—	NOT NULL	Dominio cerrado: EFECTIVO TARJETA TRANSFERENCIA.
	estado	enumerado	—	NOT NULL	Adicional razonable: PENDIENTE, EN_PREPARACION, ENTREGADO, CANCELADO (default PENDIENTE).
	created_at	fecha y hora c/ zona	—	NOT NULL	Default now().
DETALLE_PEDIDO (entidad asociativa)	id	numérico (entero)	PK	NOT NULL	Clave sustituta (justificación en Parte 2, punto 2.2).
	pedido_id	numérico (entero)	FK → PEDIDO	NOT NULL	R3: participación total.
	producto_id	numérico (entero)	FK → PRODUCTO	NOT NULL	R3, R4.
	cantidad	numérico (entero)	—	NOT NULL	R4: unidades pedidas, positiva.
	precio_unitario	numérico (monetario)	—	NOT NULL	R4: precio congelado al facturar; no negativo.
	(pedido_id, producto_id)	—	UK	—	Clave candidata natural: un producto no se repite dentro de un mismo pedido.

1.3 Relaciones, cardinalidad y participación

Relación	Tipo	Entidad / lado	Cardinalidad (mín-máx)	Participación	Justificación (regla de negocio)
CATEGORIA — PRODUCTO "agrupa"	1 : N	CATEGORIA	1 ... N	Parcial	R1: "una categoría puede tener muchos productos, o ninguno todavía (categoría recién creada)".
		PRODUCTO	1 ... 1	Total	R1: "todo producto pertenece exactamente a una categoría".
CLIENTE — PEDIDO "genera"	1 : N	CLIENTE	0 ... N	Parcial	R2: "un cliente puede no haber hecho ningún pedido aún".
		PEDIDO	1 ... 1	Total	R2: "todo pedido pertenece exactamente a un cliente registrado".
PEDIDO — PRODUCTO (a través de DETALLE_PEDIDO) "incluye / se vende en"	N : M con atributos propios	PEDIDO → DETALLE_PEDIDO	1 ... N	Total	R3 + R4: un pedido está compuesto por productos; mínimo una línea, porque un pedido sin líneas no tiene nada que preparar ni facturar.
		PRODUCTO → DETALLE_PEDIDO	0 ... N	Parcial	Un producto puede no haberse vendido nunca (recién cargado a la carta).
		DETALLE_PEDIDO	1 ... 1 a cada lado	Total	Cada línea pertenece exactamente a un pedido y a un producto; carga los atributos cantidad y precio_unitario (R4).

1.4 Preguntas guía

¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

Porque la regla **R3** es explícitamente N:M: un pedido incluye varios productos y un producto aparece en muchos pedidos. Si lo forzáramos a 1:N:

- Meter `producto_id` en PEDIDO permitiría **un solo producto por pedido** y se perdería la composición multi-ítem del pedido.
- Meter `pedido_id` en PRODUCTO permitiría **un solo pedido por producto**, lo cual contradice que la Muzzarella se vendió en los pedidos 1, 3 y 5.
- Además se pierden los **atributos de la relación (R4)**: `cantidad` y `precio_unitario` pertenecen al par (pedido, producto), no al pedido ni al producto solos. En un par 1:N esos atributos no tienen lugar físico: si el precio lo guardara producto, el pedido de marzo cambiaría cuando en abril suba el precio (violando R4).

¿Qué entidad tiene participación parcial en la relación con categoría, y cuál total? ¿Qué pasaría si invirtieras esa lectura?

En CATEGORIA — PRODUCTO: **CATEGORIA** tiene participación **parcial** (una categoría puede existir sin productos, R1: "ninguno todavía") y **PRODUCTO** tiene participación **total** (todo producto pertenece a una categoría, R1).

Si se invirtiera la lectura (categoría total y producto parcial), quedaría prohibido crear una categoría vacía —exactamente el caso que R1 menciona ("categoría recién creada")— y, a la vez, se permitiría dar de alta productos sin categoría, contradiciendo R1. El modelo invertido reflejaría las reglas de un negocio distinto (donde las categorías se crean junto con su primer producto y los productos pueden quedar huérfanos).

¿Alguno de tus atributos podría descomponerse en partes más simples? Justificá tu decisión de dejarlo compuesto o de separarlo.

Sí. El "nombre completo" del cliente se descompone en **nombre + apellido**. Justificación: permite buscar y ordenar por apellido (por ejemplo, un reporte alfabético de clientes), evita parsear cadenas compuestas y coincide con cómo se carga el dato (dos campos). Entre los atributos restantes no hay compuestos que convenga dividir: `precio`, `stock` y `cantidad` son atómicos; `email` y `telefono` se conservan como texto íntegro; `fecha` se guarda como fecha-hora con zona (no se recortan sus componentes porque se consulta en distintos formatos y zonas); `id` es una clave generada, no un dato compuesto del dominio.

Parte 2 — Derivación al modelo relacional

2.1 Tablas derivadas del ER

Reglas aplicadas: cada entidad fuerte → tabla (con su PK); cada relación **1:N** → FK del lado "1" en la tabla del lado "N"; la relación **N:M** → tabla intermedia (entidad asociativa) con FKs hacia ambos lados y los atributos de la relación. Se subraya la clave primaria; la flecha → indica clave foránea.

```

cliente      (id, nombre, apellido, email, telefono, created_at)      -- UK: email (R6)
categoria    (id, nombre, descripcion, activo, created_at)          -- UK: nombre
producto     (id, nombre, descripcion, precio, stock,                categoria_id →
categoria, activo, created_at) pedido      (id, fecha, cliente_id → cliente, forma_pago,
estado,                                     created_at)
detalle_pedido (id, pedido_id → pedido, producto_id → producto,
cantidad, precio_unitario)                -- UK: (pedido_id,
producto_id)

```

2.2 Decisión de clave en la tabla intermedia

En **detalle_pedido** la clave candidata natural es el par (`pedido_id`, `producto_id`) (regla de la planilla: un producto no se repite dentro de un mismo pedido). **Se elige igualmente una clave sustituta `id`**, manteniendo el par como restricción UNIQUE. Justificación:

- Si en el futuro otra tabla debe referenciar una línea concreta (descuentos por línea, observaciones de cocina, trazabilidad), podrá hacerlo con **una sola columna** en su FK, en lugar de dos.
- La unicidad de la combinación no se pierde: se declara UNIQUE (`pedido_id`, `producto_id`), idéntico a usar la clave compuesta como PK.
- Los joins y los índices quedan de ancho fijo y más simples.

2.3 Preguntas guía

¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?

La relación N:M quedaría corruptible desde dos frentes:

- **Huérfanos:** si `pedido_id` o `producto_id` admitieran NULL, podría existir una línea sin pedido o sin producto; PostgreSQL respeta NULL en FKs salvo que se declare NOT NULL, así que una línea así no apuntaría a nada y los recuentos (total del pedido, ventas por producto) la incluirían o la excluirían de manera inconsistente.
- **Duplicados lógicos:** por la semántica de NULL en PostgreSQL (dos NULL no chocan en una UNIQUE), si el par fuera nullable podrían existir dos filas (NULL, producto X) o (pedido Y, NULL), "duplicando" la misma asociación en la práctica.

NOT NULL + UNIQUE sobre el par garantizan que cada línea pertenece a exactamente un pedido y a exactamente un producto, y que esa asociación es única.

Si el pedido pudiera existir sin ningún producto asociado (un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1? Explica el impacto.

Sí: la participación de PEDIDO en la asociación pasaría de **total** a **parcial** (de 1..N a 0..N líneas).

Impacto en el modelo relacional: la FK detalle_pedido_pedido_id → pedido es la misma en ambos casos (siempre vive del lado de la tabla intermedia), así que no cambia la estructura de tablas. Lo que cambia es qué debemos **exigir**:

Con participación total, un pedido sin líneas es inválido; pero ninguna FK obliga a "mínimo 1 línea", y exigirlo requeriría un disparador (trigger) o lógica de aplicación.

Con participación parcial, no hace falta reforzar nada: basta con permitir que un pedido exista mientras su tabla intermedia no tenga filas para él.

En definitiva, la participación total se expresa como cardinalidad mínima ≥ 1 , un compromiso de integridad que el FK modela solo en un sentido (toda línea tiene pedido), no en ambos.

Parte 3 — Normalización hasta 3FN / BCNF

3.0 Relación universal y datos de partida

Se define la relación universal con todos los atributos de la planilla. Entre { } se marca la clave.

VentaLinea {nro_pedido, fecha, cliente, producto, categoria, precio_unitario, cantidad, subtotal, forma_pago}

nro_pedido	fecha	cliente	producto	categoria	precio_unitario	cant.	subtotal	forma_pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Paso 1 — Claves candidatas

Análisis.

- `nro_pedido` solo **no** identifica la fila: los pedidos 1 y 4 tienen más de una línea.
- `producto` solo **no**: Muzzarella aparece en los pedidos 1, 3 y 5.
- La regla "dentro de un mismo pedido, un producto no se repite en más de una línea" garantiza que **{nro_pedido, producto}** identifica una y solo una fila. Verificación en los datos: los 7 pares son distintos.

Clave candidata (única y mínima): **K = {nro_pedido, producto}** (clave primaria).

Paso 2 — Dependencias funcionales

Se listan todas las FDs que se derivan de los datos y de las reglas de negocio:

`nro_pedido` → `fecha`, `cliente`, `forma_pago` (cada pedido tiene una única fecha, un único cliente y una única forma de pago)

`producto` → `categoria` (todo producto pertenece a una única categoría — R1)

`(nro_pedido, producto)` → `precio_unitario` (el precio se congela al facturar — R4; la Muzzarella vale 1000 en el pedido 1 y 1050 en el 3)

`(nro_pedido, producto)` → `cantidad`

`(nro_pedido, producto)` → `subtotal`

`precio_unitario`, `cantidad` → `subtotal` ($\text{subtotal} = \text{precio_unitario} \times \text{cantidad}$)

Verificaciones clave en los datos: `producto` → `precio_unitario` **NO** vale (Muzzarella: 1000 y 1050); `nro_pedido` → `precio_unitario` **NO** vale (pedido 1: 1000 y 800); `cliente` → `forma_pago` **NO** vale (Ana Gómez: EFECTIVO y TRANSFERENCIA); `fecha` no determina nada (01/03 tiene Ana y Luis).

Paso 3 — Primera forma normal (1FN)

Estado: en 1FN.

Cada celda contiene un único valor atómico: no hay listas, conjuntos ni grupos repetidos (no existe una celda tipo "Muzzarella, Coca en un mismo pedido"; cada línea es una fila aparte). Los nombres compuestos ("Ana Gómez") son texto atómico: la 1FN no exige descomponer los valores semánticamente, solo prohibir valores no atómicos o repetidos. **No se requiere corrección.**

Paso 4 — Segunda forma normal (2FN)

Estado: NO está en 2FN.

Como la clave es compuesta {nro_pedido, producto}, se busca si hay atributos no clave que dependen de **solo una parte** de la clave:

Atributo no clave	Depende de	¿Parcial?	Evidencia
fecha	nro_pedido (solo)	SÍ	ignora la parte producto
cliente	nro_pedido (solo)	SÍ	ignora la parte producto
forma_pago	nro_pedido (solo)	SÍ	ignora la parte producto
categoria	producto (solo)	SÍ	ignora la parte pedido
precio_unitario	(nro_pedido, producto)	No	varía por línea; no depende de producto solo ni de pedido solo
cantidad	(nro_pedido, producto)	No	idem
subtotal	(nro_pedido, producto)	No	idem

Corrección: se separan los atributos con dependencia parcial en tablas propias. Quedan tres tablas:

```
pedido      (nro_pedido, fecha, cliente, forma_pago)      -- se extrae: depende de nro_pedido
producto    (producto, categoria)                        -- se extrae: depende de producto
linea       (nro_pedido, producto, precio_unitario,
             cantidad, subtotal)                          -- compuesta: FK a las dos anteriores
```

La 2FN queda satisfecha: pedido y producto tienen claves simples (sin dependencias parciales posibles) y en linea los atributos dependen de la clave completa.

Paso 5 — Tercera forma normal (3FN)

Estado: NO está en 3FN (una dependencia transitiva en "línea").

- **pedido (nro_pedido, fecha, cliente, forma_pago):** ningún atributo no clave determina a otro (fecha no determina cliente ni forma_pago; la misma fecha vale para Ana y Luis; Ana tiene dos formas de pago distintas). En 3FN.
- **producto (producto, categoria):** un solo atributo no clave; sin dependencia transitiva posible. En 3FN.
- **línea (nro_pedido, producto, precio_unitario, cantidad, subtotal):** se detecta la dependencia transitiva

$(\text{nro_pedido}, \text{producto}) \rightarrow \text{precio_unitario}, \text{cantidad} \rightarrow \text{subtotal}$

subtotal es un atributo no clave que depende de otros atributos no clave (precio_unitario, cantidad), con **{precio_unitario, cantidad} que NO es clave candidata**— justamente el caso que 3FN prohíbe.

Corrección: como subtotal es 100 % derivable de los otros dos ($\text{cantidad} \times \text{precio_unitario}$), se **elimina** de la tabla y se calcula al vuelo en las consultas (no se crea una tabla extra con una clave absurda). La tabla queda:

```
línea (nro_pedido, producto, precio_unitario, cantidad)
```

Paso 6 — Verificación de BCNF

BCNF exige que **todo determinante** (atributo o conjunto que determina otro) sea una clave candidata. Se revisan las FDs de cada tabla final:

Tabla	FD no trivial	¿Determinante = clave?	BCNF
pedido	nro_pedido → fecha, cliente, forma_pago	nro_pedido es la PK	✓
producto	producto → categoria	producto es la PK	✓
línea	(nro_pedido, producto) → precio_unitario, cantidad	el par es la PK	✓

No se encontró ninguna excepción: en las tres tablas el único determinante existente es la propia clave candidata. Por lo tanto el esquema está en **BCNF** (que implica 3FN).

Paso 7 — Conjunto final de tablas normalizadas

```
pedido    (nro_pedido, fecha, cliente, forma_pago)
producto  (producto, categoria)
linea     (nro_pedido, producto, precio_unitario, cantidad)
```

Pregunta de integración

Compará las tablas de la normalización con las de la Parte 2. ¿Qué entidad de tu ER corresponde a cada tabla? ¿Hay atributos de tu ER que no aparecían en esta planilla? ¿Qué te dice eso sobre modelar a partir de un enunciado vs. normalizar datos históricos?

Correspondencia:

- pedido (normalización) ↔ entidad **PEDIDO** del ER. En la planilla `cliente` es un texto embebido; en el ER es su propia entidad y el pedido guarda `cliente_id` como FK.
- producto (producto, categoria) ↔ entidades **PRODUCTO** y **CATEGORIA** del ER (la planilla mezclaba ambas en una columna "Categoría"); en el ER, producto apunta a categoría por FK.
- linea ↔ entidad asociativa **DETALLE_PEDIDO** del ER (mismos atributos: cantidad, precio_unitario).

Atributos del ER ausentes en la planilla: `cliente.email` (R6, la única clave candidata alternativa obligatoria), `cliente.telefono`, `producto.stock` (R5), `producto.precio` de lista vigente, `producto.descripcion`, `categoria.descripcion`, los indicadores `activo` (R7), las fechas de alta (`created_at`), el estado del pedido y los id numéricos.

La normalización parte de **lo que ya está registrado**: solo puede detectar dependencias presentes en los datos y no puede "inventar" atributos que nunca se relevaron (mail, stock, activo, descripciones). El modelado ER parte del **enunciado del negocio**: captura el universo completo que el sistema necesita para operar —incluso restricciones que un conjunto de datos no muestra, como la unicidad del mail o $\text{stock} \geq 0$ —. Son vías complementarias: la normalización optimiza lo existente; el ER define lo que debe existir. En este caso ambas convergen en el mismo núcleo (pedido, producto, línea), lo que valida el diseño del ER.

El atributo subtotal de cada línea es calculable como $\text{cantidad} \times \text{precio_unitario}$. ¿Conviene almacenarlo igual, o recalcularlo al vuelo? ¿Esa decisión viola alguna forma normal, o es un tema distinto (rendimiento, trazabilidad)?

Conviene no almacenarlo y calcularlo al vuelo ($\text{cantidad} * \text{precio_unitario}$). Almacenarlo **sí viola 3FN**: introduciría la dependencia transitiva $(\text{nro_pedido}, \text{producto}) \rightarrow \text{precio_unitario}, \text{cantidad} \rightarrow \text{subtotal}$, con $\{\text{precio_unitario}, \text{cantidad}\}$ que no es clave candidata. No es un tema de trazabilidad: lo que la historia debe preservar es `precio_unitario` (porque el precio de lista cambia, R4) y eso ya se guarda por línea; el subtotal se puede reproducir idéntico en cualquier momento. Como se trata de una multiplicación, tampoco hay argumento real de rendimiento que justifique la desnormalización.

Anexo — Parte 4 (esquema DDL)

El resultado conciliado (Partes 2 + 3) se materializó en `schema.sql`, adjunto al entregable. Incluye: `CREATE TYPE` para forma de pago y estado de pedido; claves primarias con `BIGINT GENERATED ALWAYS AS IDENTITY`; claves foráneas `REFERENCES` con `ON DELETE RESTRICT` justificado (R7); `UNIQUE` sobre `cliente.email` (R6) y sobre el par de la tabla intermedia; cuatro restricciones `CHECK` ($\text{precio} \geq 0$, $\text{stock} \geq 0$, $\text{cantidad} > 0$, $\text{precio_unitario} \geq 0$); `NOT NULL` según la participación definida en la Parte 1; `DEFAULT` (`activo`, `created_at`, `fecha`, `stock`, `estado`); y tres índices comentados según el uso esperado. El subtotal no se almacena